

INTERNET OF THINGS (IoT)

Unit III — Comprehensive Study Notes

Networking and Communication Protocols

TY BSC IT | Exam Preparation Guide

Cognitive Level: K1, K2, K3, K4, K6 | Hours: 12 | PO: 1, 2, 3, 4, 5

- ✓ Wired and Wireless Protocols — UART, SPI, I2C, Bluetooth, Wi-Fi, Zigbee, LoRa
 - ✓ Application Layer Protocols — MQTT, CoAP, HTTP/HTTPS for IoT
 - ✓ Network Layer and Application Layer Protocols
 - ✓ Node-to-Gateway-to-Cloud Communication Architecture

1. Wired Communication Protocols

Wired communication protocols define the rules for how data is transmitted between devices over physical electrical connections. In IoT, wired protocols are used primarily for short-range, board-level communication between microcontrollers, sensors, and peripheral devices.

1.1 UART (Universal Asynchronous Receiver/Transmitter)

UART — Definition

UART is a hardware communication protocol that enables asynchronous serial communication between two devices using just two data lines: TX (Transmit) and RX (Receive). 'Asynchronous' means there is NO shared clock signal — both devices must agree on the same baud rate (data speed) in advance.

1.1.1 UART — Key Characteristics & Working

UART Property	Details
Communication Type	Asynchronous serial — no clock line shared between devices
Wires Required	2 data wires (TX and RX) + GND. TX of device A connects to RX of device B and vice versa.
Topology	Point-to-point only — connects exactly ONE device to ONE device
Baud Rate	Bits per second agreed upon by both devices. Common: 9600, 19200, 38400, 57600, 115200 bps
Data Frame	1 Start bit (LOW) + 5–9 Data bits + Optional Parity bit + 1–2 Stop bit(s) (HIGH)
Start Bit	Logic LOW (0) — signals the beginning of a new data byte
Stop Bit	Logic HIGH (1) — signals end of data byte, line returns to idle
Parity Bit	Optional error detection: None, Even, or Odd parity. Rarely used in modern UART.
Full/Half Duplex	Full-duplex — both devices can transmit and receive simultaneously (separate TX/RX lines)
Voltage Levels	Standard UART: 0V = Logic 0, 3.3V or 5V = Logic 1. RS-232 uses $\pm 12V$ for longer distances.
Max Speed	Typically up to 115200 bps (standard), some implementations up to 1 Mbps+
Max Distance	~15 metres for standard TTL UART; RS-232 ($\pm 12V$) extends to ~15m; RS-485 up to 1200m

Arduino UART Pins	D0 (RX), D1 (TX) — shared with USB. Arduino Mega has 4 UART ports (Serial, Serial1, Serial2, Serial3)
--------------------------	---

1.1.2 UART Data Frame Diagram

UART Data Frame Structure

IDLE START D0 D1 D2 D3 D4 D5 D6 D7 STOP IDLE
HIGH | LOW | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | HIGH | HIGH

- IDLE state: Line stays HIGH (logic 1) when no data is being sent
- START bit: Always a single LOW (logic 0) bit — alerts receiver that a byte is coming
- DATA bits: 8 data bits (LSB first in most UART implementations) — the actual byte value
- STOP bit: Always HIGH (logic 1) — marks end of byte, line returns to idle
- The receiver samples each bit in the MIDDLE of its time slot (based on agreed baud rate timing)

1.1.3 UART Applications in IoT

- **GPS Modules (NEO-6M/NEO-8M):** GPS sends NMEA sentences at 9600 baud via UART. Arduino reads latitude/longitude data.
- **Bluetooth Modules (HC-05/HC-06):** Classic Bluetooth serial bridge. AT commands configure the module; data passes transparently via UART at 9600 or 38400 baud.
- **GSM/4G Modules (SIM800L, SIM7600):** AT command interface over UART. Send SMS, make calls, or connect to the internet for IoT data transmission.
- **Arduino Serial Monitor:** USB-to-UART bridge (ATmega16U2) allows debug output via Serial.print() to PC at any baud rate.
- **Inter-Arduino Communication:** Two Arduinos can communicate by connecting TX of one to RX of the other and sharing GND.

1.2 SPI (Serial Peripheral Interface)

SPI — Definition

SPI is a synchronous, full-duplex, master-slave serial communication protocol developed by Motorola. It uses a shared clock signal (SCK) generated by the master to synchronize data transfer. SPI is the fastest of the common short-range communication protocols and uses 4 wires: MOSI, MISO, SCK, and SS (Chip Select).

1.2.1 SPI Signals — Detailed Explanation

SPI Signal	Full Name, Direction & Function
MOSI	Master Out Slave In — carries data FROM the master TO the slave device. Master drives this line.

MISO	Master In Slave Out — carries data FROM the slave BACK TO the master. Slave drives this line.
SCK (SCLK)	Serial Clock — generated ONLY by the master. Synchronizes data transmission. Data bits are sampled on clock edges.
SS / CS	Slave Select / Chip Select — active LOW signal. Master pulls a specific slave's SS pin LOW to select that device for communication. Each additional SPI device needs its own dedicated SS pin on the master.

1.2.2 SPI Communication — How it Works

1. Master pulls the SS/CS pin of the target slave LOW (activates it).
2. Master generates clock pulses on SCK line.
3. On each clock edge, master sends 1 bit on MOSI and simultaneously receives 1 bit on MISO.
4. Data is shifted out MSB first (typically) — one full byte transferred in 8 clock cycles.
5. After transfer, master pulls SS HIGH to deselect the slave.

1.2.3 SPI Modes — Clock Polarity (CPOL) and Phase (CPHA)

SPI has 4 modes defined by two parameters that determine when data is sampled and shifted:

Mode	CPOL	CPHA	Clock Idle State	Data Sampled On	Common Usage
SPI Mode 0	0	0	LOW	Rising edge	Most common — used by many sensors (MCP3208, SD cards)
SPI Mode 1	0	1	LOW	Falling edge	Some sensors and displays
SPI Mode 2	1	0	HIGH	Falling edge	Less common
SPI Mode 3	1	1	HIGH	Rising edge	Some ADCs, RTC chips (DS1302)

SPI Property	Value / Details
Speed	Very fast — up to 10 MHz+ for many devices; some run at 50–80 MHz
Duplex	Full-duplex — data flows in both directions simultaneously on MOSI and MISO
Wires (minimum)	4 (MOSI + MISO + SCK + SS per slave)
Multi-slave	Each additional slave needs one dedicated SS pin on the master — becomes pin-hungry with many devices
Arduino SPI pins	UNO: D11(MOSI), D12(MISO), D13(SCK), D10(SS). MEGA: D51(MOSI), D50(MISO), D52(SCK), D53(SS)

Library	SPI.h (Arduino) — SPI.begin(), SPI.transfer(byte), SPI.end()
Applications	SD cards, TFT/LCD displays, W5100 Ethernet, MAX6675 thermocouple, nRF24L01 radio, DAC chips
Limitation	No built-in device addressing (unlike I2C) — needs separate SS wire per device

1.3 I2C (Inter-Integrated Circuit) — Deep Dive

I2C — Definition

I2C (pronounced 'I-squared-C' or 'I-two-C') is a synchronous, multi-master, multi-slave, serial communication bus developed by Philips Semiconductor (1982). It uses only TWO bidirectional wires — SDA (Serial Data) and SCL (Serial Clock) — and allows up to 127 devices to share a single bus, each identified by a unique 7-bit address.

1.3.1 I2C Bus — Detailed Working

- Both SDA and SCL lines are pulled HIGH by external pull-up resistors (typically 4.7 kΩ to VCC).
- Master generates a START condition: SDA transitions HIGH → LOW while SCL is HIGH.
- Master sends a 7-bit slave address followed by a READ/WRITE bit (0=write, 1=read).
- The addressed slave acknowledges by pulling SDA LOW for one clock pulse (ACK).
- Data bytes are transferred (8 bits each), each followed by an ACK from the receiver.
- Master generates a STOP condition: SDA transitions LOW → HIGH while SCL is HIGH — releases the bus.

I2C Feature	Detailed Explanation
SDA (Serial Data)	Bidirectional open-drain data line. Carries address and data bits. Both master and slave can drive this line LOW; pull-up resistor brings it HIGH when idle.
SCL (Serial Clock)	Clock line driven by the master. Data on SDA is stable (valid) while SCL is HIGH; changes on SDA occur while SCL is LOW.
Pull-up Resistors	REQUIRED — typically 4.7 kΩ to VCC. Without pull-ups, the I2C bus will not work. Many sensor breakout boards include these (look for R1/R2 on the PCB).
Device Addressing	7-bit address (0–127, but 16 reserved). 112 usable unique addresses. Some devices allow changing address bits via solder jumpers or ADDR pins (e.g., BMP280: 0x76 or 0x77).
Speed Modes	Standard: 100 kbps Fast: 400 kbps Fast-Plus: 1 Mbps High-Speed: 3.4 Mbps Ultra-Fast: 5 Mbps (write-only)
ACK / NACK	After each byte, receiver pulls SDA LOW = ACK (acknowledge, 'I received it'). If SDA stays HIGH = NACK (error or end of read).

Bus Arbitration	Multiple masters can be on the bus. If two masters transmit simultaneously, the one sending a LOW when the other sends HIGH loses arbitration and stops.
Repeated Start	Master can send a new address without sending STOP first — allows atomic read-after-write sequences without releasing the bus.
Wire.h Library	Arduino: Wire.begin(), Wire.beginTransmission(addr), Wire.write(data), Wire.endTransmission(), Wire.requestFrom(addr, bytes), Wire.read()
RPi Python Library	smbus2 or smbus: bus = SMBus(1), bus.read_byte_data(addr, reg), bus.write_byte_data(addr, reg, val)
Common I2C Devices	MPU-6050 IMU (0x68), BMP280 barometer (0x76), SSD1306 OLED (0x3C), PCF8574 I/O expander (0x20), DS3231 RTC (0x68), ADS1115 ADC (0x48)

1.3.2 Wired Protocol Comparison Table

Feature	UART	SPI	I2C
Wires	2 (TX, RX)	4 (MOSI,MISO,SCK,SS)	2 (SDA, SCL)
Clock	None (async)	SCK (master drives)	SCL (master drives)
Addressing	None (point-to-point)	Via SS pin per device	7-bit address per device
Max Devices	1 (point-to-point)	Unlimited (1 SS per slave)	127 on one bus
Speed	Up to ~1 Mbps	Up to 50 MHz+	100 kbps – 5 Mbps
Duplex	Full-duplex	Full-duplex	Half-duplex
Complexity	Very simple	Simple	Moderate (needs addressing)
Pull-ups needed	No	No	YES — 4.7 kΩ required
Distance	Up to 15m (RS-232)	Short (<30 cm board-level)	Short (<1m typical)
Best For	Module-to-module long-range serial	High-speed data (display, SD)	Multiple sensors on 2 wires

2. Wireless Communication Protocols

Wireless protocols enable IoT devices to communicate without physical cables — essential for mobile, remote, and large-scale IoT deployments. Each wireless protocol is engineered with a specific trade-off between range, data rate, power consumption, and cost.

2.1 Bluetooth & Bluetooth Low Energy (BLE)

Bluetooth — Definition

Bluetooth is a short-range wireless technology standard (IEEE 802.15.1) for exchanging data between fixed and mobile devices over short distances using UHF radio waves in the 2.4 GHz ISM band. It was designed for low-power, low-cost wireless communication to replace short cables.

2.1.1 Bluetooth Classic vs Bluetooth Low Energy (BLE)

Feature	Bluetooth Classic (BR/EDR)	Bluetooth Low Energy (BLE)
Full Name	Bluetooth Basic Rate/Enhanced Data Rate	Bluetooth Low Energy (Bluetooth 4.0+)
Data Rate	1–3 Mbps	125 kbps – 2 Mbps (BLE 5.0)
Range	~10–100 metres	~10–400 metres (BLE 5.0 Long Range)
Power Consumption	High (~30 mA active)	Very Low (~1–15 mA active, μ A in sleep)
Latency	~100 ms	~3–6 ms (much lower)
Connection Setup	Slower pairing process	Instant connection (~3 ms)
Battery Life	Hours to days	Months to years on coin cell battery
Data Profile	Continuous streaming (audio, file transfer)	Small periodic data packets (sensor data)
IoT Suitability	Moderate — audio, some data transfer	High — ideal for IoT sensors and wearables
Arduino Module	HC-05 (master/slave), HC-06 (slave only)	HM-10, CC2541 BLE modules
RPi Built-in	Yes — RPi 3/4/5 include Bluetooth Classic + BLE	Yes — same chip
Applications	Wireless headphones, speakers, file transfer	Smartwatches, fitness bands, beacons, medical devices

2.1.2 Bluetooth Architecture — Piconet & Scatternet

- **Piconet:** The basic Bluetooth network unit. One master device + up to 7 active slave devices (255 parked/inactive). All slaves synchronize to the master's clock.
- **Scatternet:** Multiple overlapping piconets. A device can be a slave in one piconet and a master in another — enables extended networking.
- **Frequency Hopping Spread Spectrum (FHSS):** Bluetooth hops between 79 different 1MHz channels in the 2.4 GHz band 1600 times per second — reduces interference and improves security.

2.1.3 BLE GATT Profile (for IoT)

GATT (Generic Attribute Profile): Defines how BLE devices exchange data. Uses a hierarchical structure:

- **Profile:** Collection of services defining a use case (e.g., Heart Rate Profile)
- **Service:** Groups related characteristics (e.g., Heart Rate Service UUID: 0x180D)
- **Characteristic:** A specific data point with a UUID, value, and properties (Read, Write, Notify)
- **Descriptor:** Metadata about a characteristic (e.g., human-readable name, units)
- **Central (Client):** Scans for and connects to BLE devices — typically a phone or gateway
- **Peripheral (Server):** Advertises its presence and provides data — typically an IoT sensor

2.2 Wi-Fi (IEEE 802.11)

Wi-Fi — Definition

Wi-Fi (Wireless Fidelity) is a family of wireless network protocols based on the IEEE 802.11 standards for wireless local area networking (WLAN). Wi-Fi allows devices to connect to the Internet via a wireless access point (router). It is the dominant wireless protocol for high-bandwidth IoT applications.

2.2.1 Wi-Fi Standards Evolution

Standard	Wi-Fi Gen.	Frequency	Max Speed	Range (indoor)	IoT Relevance
802.11b	Wi-Fi 1	2.4 GHz	11 Mbps	~35m	Legacy — rarely used
802.11g	Wi-Fi 3	2.4 GHz	54 Mbps	~38m	Legacy
802.11n	Wi-Fi 4	2.4/5 GHz	600 Mbps	~70m	Still widely used — ESP8266 uses this
802.11ac	Wi-Fi 5	5 GHz	3.5 Gbps	~35m	RPi 3B+/4 — smartphones, smart home
802.11ax	Wi-Fi 6	2.4/5/6 GHz	9.6 Gbps	~35m	Dense IoT deployments — OFDMA

802.11ah	HaLow	Sub-1 GHz	347 Mbps	~1 km	Designed for IoT — long range, low power
----------	-------	-----------	----------	-------	--

2.2.2 Wi-Fi Architecture for IoT

Wi-Fi Concept	Explanation
SSID	Service Set Identifier — the name of the Wi-Fi network (max 32 characters). Devices connect to an SSID.
Access Point (AP)	Router or dedicated AP — connects Wi-Fi devices to the wired network and Internet. Central node in infrastructure mode.
Infrastructure Mode	Client devices connect to an AP. The AP routes traffic. Standard mode for home/office Wi-Fi.
Ad-hoc / Mesh Mode	Devices connect directly to each other without an AP. Used in mesh networking (like OpenWrt mesh, ESP-MESH).
Frequency Bands	2.4 GHz: Better range (longer wavelength), more interference (crowded band), fewer channels. 5 GHz: Faster, less interference, shorter range.
DHCP	Router assigns IP addresses automatically. IoT devices typically receive dynamic IP unless static IP is configured.
Station Mode (STA)	IoT device connects to an existing Wi-Fi network (like a normal client). ESP8266/ESP32 default mode.
Access Point Mode (AP)	IoT device BECOMES a Wi-Fi access point itself. Other devices connect to it. ESP8266/ESP32 can do this.
Dual Mode (AP+STA)	ESP8266/ESP32 can simultaneously be a station (connected to router) AND an access point (for configuration portal).
Security	WPA3 (latest), WPA2-PSK (current standard), WEP (deprecated — insecure). Always use WPA2 or WPA3 for IoT.

2.2.3 Wi-Fi for IoT — ESP8266 / ESP32

ESP8266 (NodeMCU): Single-core 80MHz, 802.11 b/g/n Wi-Fi, 80KB RAM, programmable with Arduino IDE. Extremely popular low-cost (~₹150) Wi-Fi IoT module.

ESP32: Dual-core 240MHz, Wi-Fi 802.11 b/g/n + BLE 4.2, 520KB RAM, 4MB Flash. The dominant IoT platform (~₹300–500).

```
// ESP32/ESP8266 connecting to Wi-Fi — Arduino IDE
#include <WiFi.h>    // ESP32    (or #include <ESP8266WiFi.h> for ESP8266)

const char* ssid     = "YourNetworkName";
const char* password = "YourPassword";
```

```

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);          // Start connection attempt
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");      // Wait until connected
  }
  Serial.println("\nConnected! IP: " + WiFi.localIP().toString());
}

```

2.3 Zigbee (IEEE 802.15.4)

Zigbee — Definition

Zigbee is a low-power, low-data-rate wireless mesh networking protocol based on the IEEE 802.15.4 standard. It operates in the 2.4 GHz band and is designed specifically for applications requiring battery-powered sensor networks with long battery life, reliability, and self-healing mesh topology.

Zigbee Property	Details
Frequency	2.4 GHz (global), 868 MHz (Europe), 915 MHz (USA)
Data Rate	250 kbps at 2.4 GHz (very low — suitable for small sensor payloads only)
Range	10–100 metres per hop; mesh extends this to hundreds of metres or more
Power Consumption	Very low — end devices can sleep (μ A) and run for years on AA batteries
Network Topology	Mesh, Star, or Tree — mesh is the key advantage over Bluetooth/Wi-Fi
Max Devices	Up to 65,000 devices in a Zigbee network (vs 7 for Bluetooth Classic)
Security	AES-128 encryption built in
Latency	~15–30 ms — suitable for control applications
Standard	Zigbee Alliance (now CSA — Connectivity Standards Alliance) maintains the spec
Relation to Matter	Zigbee forms the basis of the Matter standard (used by Apple HomeKit, Google Home, Amazon Alexa)

2.3.1 Zigbee Network Roles

- **Zigbee Coordinator (ZC):** ONE per network. Starts the network, assigns addresses, stores network info. Usually the most powerful device (e.g., a hub or gateway). Acts as the root of the network tree.
- **Zigbee Router (ZR):** Mains-powered devices that forward (route) messages between devices. Extend the mesh range. Examples: smart light bulbs (always powered).
- **Zigbee End Device (ZED):** Battery-powered leaf nodes. Only communicate with their parent (router or coordinator). Sleep most of the time to conserve power. Examples: door sensors, temperature sensors, motion detectors.

2.3.2 Zigbee Mesh Networking — Key Advantage

Why Zigbee Mesh is Powerful for IoT

In a mesh network, each router-capable Zigbee device can relay messages for other devices.

If one route between Device A and the Coordinator is blocked (device fails), the mesh automatically finds an alternate route — self-healing.

This makes Zigbee networks HIGHLY RELIABLE for building automation where device failures are common.

Each hop extends the network range. A network with 5 routers each covering 30m can span 150m+.

Real-world use: Smart buildings with hundreds of light switches and sensors all communicating through a mesh to one Zigbee gateway.

Applications: Smart home automation (Philips Hue ZigBee, IKEA TRADFRI, SmartThings), building automation (HVAC, lighting control), industrial sensor networks, smart metering, healthcare monitoring.

2.4 LoRa and LoRaWAN

LoRa and LoRaWAN — Definitions

LoRa (Long Range) is a spread-spectrum modulation technique developed by Semtech that enables long-range wireless communication with very low power. LoRaWAN (LoRa Wide Area Network) is the MAC-layer protocol and network architecture built on top of LoRa modulation — defining how devices communicate with gateways and cloud servers.

2.4.1 LoRa Physical Layer

- **Modulation:** CSS (Chirp Spread Spectrum) — uses chirp signals that sweep across a bandwidth. Highly resistant to multipath and Doppler effects.
- **Spreading Factor (SF):** SF7 to SF12. Higher SF = longer range, lower data rate, more airtime. Lower SF = faster, shorter range. SF12 can reach 15+ km in rural areas.
- **Bandwidth:** 125 kHz, 250 kHz, or 500 kHz. Wider BW = faster data rate, shorter range.
- **Coding Rate (CR):** 4/5 to 4/8. Higher CR = more redundancy = better interference resistance but lower throughput.

- **Frequency Bands:** 868 MHz (EU), 915 MHz (USA/Australia), 433 MHz (Asia). Sub-GHz frequencies penetrate buildings better than 2.4 GHz.

2.4.2 LoRaWAN Network Architecture

LoRaWAN Component	Role & Description
End Nodes (Devices)	Battery-powered IoT devices with LoRa radio. Send sensor data uplink to gateways. Receive commands downlink. Can be Class A, B, or C devices.
LoRa Gateway	Receives LoRa transmissions from ALL end nodes in range (no pairing needed). Forwards ALL received packets to the network server via standard IP connection (Ethernet/4G/Wi-Fi). One gateway can serve thousands of end nodes.
Network Server (LNS)	Central server (cloud or on-premise) that handles: deduplication of packets (multiple gateways may receive same packet), MAC commands, ADR (Adaptive Data Rate), routing to application servers.
Application Server	Receives processed sensor data from network server. Stores, analyzes, and visualizes data. Sends commands back down. Examples: AWS IoT Core, The Things Network (TTN) application server.
Join Server	Manages device authentication for OTAA (Over The Air Activation) — the secure device join procedure.

2.4.3 LoRaWAN Device Classes

Class	Receive Windows	Power Consumption	Latency (Downlink)	Best For
Class A	2 short RX windows after each uplink ONLY	Lowest — sleeps between uplinks	High (waits for next uplink)	Most battery-powered sensors (soil, temperature)
Class B	Scheduled RX windows (beaconed, periodic)	Medium	Medium (predictable)	Actuators needing periodic control (valves, switches)
Class C	Continuously listening (except when transmitting)	Highest (near-continuous power)	Very Low (almost immediate)	Mains-powered devices (smart meters, actuators)

LoRaWAN Property	Typical Value
Range (rural / open)	10–15 km (LoRa SF12, 125kHz BW)
Range (urban)	2–5 km (buildings attenuate signal)

Data Rate	0.3–50 kbps (inversely related to range via SF)
Battery Life	5–10+ years on AA batteries (Class A with hourly uplinks)
Payload Size	Maximum 51–242 bytes per message (SF dependent) — LoRaWAN is NOT for large data
Security	AES-128 encryption at two layers: NwkSKey (network) and AppSKey (application)
Activation Methods	OTAA (Over-The-Air Activation — secure, preferred) or ABP (Activation By Personalization — hardcoded keys, less secure)
Popular Platforms	The Things Network (TTN — free community network), ChirpStack (open-source), AWS IoT Core for LoRaWAN
Common Modules	SX1276/SX1278 (Semtech chip), TTGO LoRa32, Heltec WiFi LoRa 32, RAK WisNode

2.4.4 LoRaWAN vs Other LPWAN Technologies

Property	LoRaWAN	NB-IoT	Sigfox	LTE-M (Cat-M1)
Spectrum	Unlicensed (free ISM band)	Licensed (cellular)	Unlicensed	Licensed (cellular)
Range	~15 km rural	~10 km	~50 km rural	~10 km
Data Rate	0.3–50 kbps	60–250 kbps	100 bps (uplink!)	Up to 1 Mbps
Power	Ultra-low (years on battery)	Ultra-low	Ultra-low	Low
Cost	Very low — free community networks	Monthly carrier fee	Monthly carrier fee	Monthly carrier fee
Best For	Long-range sensors, smart agriculture	Smart meters, urban IoT	Ultra-simple trackers	Voice + data IoT

3. Application Layer Protocols for IoT

Application layer protocols define how IoT devices and applications exchange data over a network. Three protocols dominate IoT application communication: MQTT (pub/sub for constrained devices), CoAP (REST for constrained devices), and HTTP/HTTPS (REST for capable devices and cloud APIs).

3.1 MQTT (Message Queuing Telemetry Transport) — Complete Coverage

MQTT — Formal Definition

MQTT is a lightweight, binary, publish-subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency networks. It runs over TCP/IP (port 1883 for unencrypted, port 8883 for TLS/SSL encrypted). Designed by Andy Stanford-Clark (IBM) and Arlen Nipper in 1999 to monitor oil pipelines via satellite links. Standardized by OASIS in 2014 as MQTT 3.1.1; MQTT 5.0 released in 2019.

3.1.1 MQTT Core Concepts — Deep Dive

MQTT Concept	Detailed Explanation
Broker	Central server that receives ALL messages from publishers and routes them to all matching subscribers. The intelligence of the MQTT system. Popular brokers: Eclipse Mosquitto (open-source, runs on RPi), HiveMQ, EMQX, AWS IoT Core, Google Cloud IoT Core.
Client	Any device or application that connects to the broker. A client can be BOTH a publisher and subscriber simultaneously. IoT sensor = publisher. Mobile app = subscriber. Dashboard = subscriber. Backend server = both.
Topic	A UTF-8 string used to route messages. Hierarchical structure using '/' as separator. Example: 'home/floor1/bedroom/temperature'. Think of it as an address for messages.
Payload	The actual data content of the MQTT message. MQTT is payload-agnostic — can be any format: plain text ('24.5'), JSON ({"temp":24.5, "unit":"C"}), binary, or XML. No schema enforced by MQTT itself.
Publish	MQTT PUBLISH command — client sends a message to a topic. Publisher does NOT know who is subscribed.
Subscribe	MQTT SUBSCRIBE command — client registers interest in a topic (or topic pattern with wildcards). Will receive all future messages published to matching topics.

CONNECT/DISCONNECT	Clients send CONNECT to establish session (clientId, username, password, keepAlive). DISCONNECT cleanly closes the connection.
PINGREQ/PINGRESP	Keep-alive mechanism. Client sends PINGREQ if no message activity for keepAlive interval. Broker responds PINGRESP. If no response, connection considered dead.

3.1.2 MQTT Topic Wildcards — Critical for Exams



MQTT Topic Wildcards

Single-Level Wildcard (+) — matches EXACTLY ONE topic level.

Example: 'home+/temperature' matches:

- ✓ home/bedroom/temperature
- ✓ home/kitchen/temperature
- ✗ home/floor1/bedroom/temperature (two levels after home/)

Multi-Level Wildcard (#) — matches ANY number of remaining levels. Must be the LAST character.

Example: 'home/#' matches:

- ✓ home/bedroom/temperature
- ✓ home/floor1/bedroom/temperature
- ✓ home/kitchen/light/status
- ✓ home/ (empty subtopic)

'\$SYS/#' — special broker system topic prefix for broker statistics (uptime, connected clients, etc.)

3.1.3 MQTT QoS (Quality of Service) Levels — Exam Favourite

QoS Level	Name	Guarantee	Protocol Exchange	Overhead	When to Use
QoS 0	At Most Once	NO guarantee — fire and forget. Message may be lost if network fails.	PUBLISH (no ACK)	Lowest	Non-critical, frequent data (temperature every 5s) — some loss acceptable
QoS 1	At Least Once	Message DEFINITELY delivered but may be DUPLICATED (if ACK is lost,	PUBLISH → PUBACK	Medium	Sensor alerts, commands — delivery essential,

		sender retransmits).			can handle duplicates
QoS 2	Exactly Once	Message delivered EXACTLY once. No loss, no duplicates. Four-way handshake.	PUBLISH→PUBREC→PUBREL→PUBCOMP	Highest	Financial transactions, critical commands where duplicates cause harm

3.1.4 MQTT Retained Messages & Last Will Testament

MQTT Feature	Explanation & Use Case
Retained Message	RETAIN flag = TRUE on a PUBLISH. Broker stores the LAST retained message for each topic. When a new subscriber subscribes to that topic, they IMMEDIATELY receive the last retained message — even if the publisher hasn't sent anything recently. Use case: Store the current state of an actuator (light ON/OFF) so new subscribers know the current state instantly.
Last Will & Testament (LWT)	Client specifies a 'Will Message' in its CONNECT packet (topic + payload + QoS + retain). If the client UNEXPECTEDLY disconnects (network failure, power loss — NOT a clean DISCONNECT), the broker automatically publishes the Will Message on the specified topic. Use case: A sensor publishes 'status/sensor1 OFFLINE' to notify all subscribers that it has died. If normal, it would publish 'ONLINE' before clean disconnect.
Clean Session	If CleanSession=TRUE (default), broker discards all subscriptions and queued messages when client disconnects. If CleanSession=FALSE (persistent session), broker stores subscriptions and QoS 1/2 messages while client is offline — delivered when client reconnects.
Client ID	Unique string identifying the MQTT client to the broker. If two clients use the same ID, the second connection kicks out the first — a common source of disconnect loops in IoT.

3.1.5 MQTT Implementation — Arduino ESP32 Example

```
#include <WiFi.h>
#include <PubSubClient.h>    // Install via Library Manager

const char* ssid      = "HomeWiFi";
const char* password  = "password123";
const char* broker    = "broker.hivemq.com";  // Public test broker
const int  port       = 1883;
```



```

WiFiClient  espClient;
PubSubClient mqtt(espClient);

// Callback: called when a subscribed message arrives
void onMessage(char* topic, byte* payload, unsigned int length) {
    String msg = "";
    for (int i = 0; i < length; i++) msg += (char)payload[i];
    Serial.println "[" + String(topic) + " ] " + msg;
    if (msg == "ON")  digitalWrite(LED_BUILTIN, HIGH);
    if (msg == "OFF") digitalWrite(LED_BUILTIN, LOW);
}

void reconnect() {
    while (!mqtt.connected()) {
        if (mqtt.connect("ESP32_Device")) {           // Connect with
clientID
            mqtt.subscribe("home/led/control");       // Subscribe to
control topic
        } else { delay(5000); }
    }
}

void setup() {
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) delay(500);
    mqtt.setServer(broker, port);
    mqtt.setCallback(onMessage);
}

void loop() {
    if (!mqtt.connected()) reconnect();
    mqtt.loop();    // MUST be called – processes incoming messages
    // Publish sensor data every 5 seconds
    static unsigned long last = 0;
    if (millis() - last > 5000) {
        last = millis();
        String payload = "{\"temp\":\"" + String(random(20,30)) + "\"}";
        mqtt.publish("home/temperature", payload.c_str()); // Publish to
topic
    }
}

```

3.2 CoAP (Constrained Application Protocol)

CoAP — Definition

CoAP is a lightweight RESTful application-layer protocol designed by the IETF (RFC 7252) specifically for constrained IoT devices (microcontrollers with limited RAM/CPU) and constrained networks (low bandwidth, high packet loss). It uses UDP (not TCP) for transport and follows the same GET/POST/PUT/DELETE model as HTTP but with a tiny 4-byte header.

3.2.1 CoAP Architecture & Features

CoAP Feature	Detailed Explanation
Transport Protocol	UDP (User Datagram Protocol) — connectionless, no handshake, minimal overhead. HTTP uses TCP which requires a 3-way handshake (expensive for constrained devices).
Header Size	4 bytes fixed header — vs HTTP headers of 500+ bytes. This tiny overhead is critical for devices with small RAM.
REST Model	Uses same 4 methods as HTTP: GET (retrieve), POST (create), PUT (update), DELETE (remove). Resources addressed by URI: <code>coap://device.local/sensors/temp</code>
Message Types	CON (Confirmable — reliable, requires ACK), NON (Non-Confirmable — unreliable, no ACK), ACK (acknowledgement of CON), RST (reset, reject a message)
Reliability over UDP	CoAP adds its own reliability for CON messages: sender retransmits with exponential back-off until ACK received. ACK contains the message ID.
Response Codes	Similar to HTTP: 2.xx (Success: 2.01 Created, 2.04 Changed, 2.05 Content), 4.xx (Client Error: 4.04 Not Found), 5.xx (Server Error)
Observe Extension	RFC 7641 — client subscribes to a resource and receives updates when the resource state changes. Bridges toward pub/sub. Similar to MQTT subscription but request-response based.
Block-wise Transfer	RFC 7959 — allows large data to be split into blocks for transfer over limited-MTU networks.
Discovery	Multicast address (<code>coap://[FF02::FD]</code>) allows clients to discover available CoAP servers on the network without prior configuration.
Security	DTLS (Datagram TLS) for encrypted CoAP — equivalent to TLS/SSL for HTTP but designed for UDP.
Port	UDP port 5683 (unencrypted CoAP), UDP port 5684 (DTLS-secured CoAP)

3.2.2 CoAP Message Format

CoAP Packet Structure (minimum 4 bytes)

Byte 1: Ver(2 bits) | T — Type(2 bits) | TKL — Token Length(4 bits)

Byte 2: Code (Request method or Response code) — e.g., 0.01=GET, 0.02=POST, 2.05=Content

Bytes 3–4: Message ID (16 bits) — used to match ACKs with CON messages

Bytes 5+: Token (0–8 bytes, length given by TKL) — matches requests to responses

Then: Options (URI path, content format, etc.) encoded in delta TLV format

Then: 0xFF marker byte (if payload follows)

Then: Payload (actual data)

Example GET request: Ver=1, T=CON, Code=GET(0.01), ID=1234, Token=0xAB, Option: Uri-Path='temperature'

3.2.3 CoAP vs MQTT — When to Choose Which

Aspect	CoAP	MQTT
Communication Model	Request-Response (REST)	Publish-Subscribe
Transport	UDP (connectionless)	TCP (connection-oriented)
Header Overhead	4 bytes minimum	2 bytes minimum (MQTT also tiny)
Reliability	Optional (CON/NON messages)	Guaranteed via QoS 1 and 2
Broker Required	NO — direct device-to-device	YES — broker is mandatory
Discovery	Built-in multicast discovery	No built-in discovery
Bidirectional Push	Via Observe extension	Native — subscribers get pushed data
Many-to-Many	Not native	Native via topics
Routing through NAT	Difficult (UDP + NAT)	Easier (TCP maintains connection)
Best For	Device-to-device in local network, resource-constrained devices	IoT cloud platforms, telemetry to cloud, real-time dashboards

3.3 HTTP and HTTPS for IoT

HTTP — Definition

HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the World Wide Web. In IoT, HTTP provides a standard, well-understood request-response protocol for sending sensor data to cloud APIs and receiving commands. HTTPS adds TLS/SSL encryption for secure communication.

3.3.1 HTTP in IoT Context

HTTP Concept	IoT Application
HTTP GET	Retrieve current sensor value from IoT device's web server. Example: HTTP GET http://device.local/temperature → returns {"value": 24.5}
HTTP POST	Send sensor data TO a cloud API endpoint. Example: POST /api/v1/data with JSON body {"temperature": 24.5, "deviceId": "sensor01"}
HTTP PUT	Update a resource/configuration. Example: PUT /api/v1/devices/LED1 with body {"state": "ON"}
HTTP DELETE	Remove a resource. Example: DELETE /api/v1/devices/sensor01 — remove device from cloud registry.
REST API	Most IoT cloud platforms (ThingSpeak, Blynk, Ubidots, AWS IoT) provide REST APIs over HTTPS for data upload/retrieval.
Polling	IoT device periodically sends HTTP GET to check for new commands. Inefficient (many empty responses) but simple.
WebHook	Cloud server sends HTTP POST to IoT device/gateway when an event occurs — reverses the request direction for near-real-time push.
Status Codes	200 OK, 201 Created, 400 Bad Request, 401 Unauthorized (API key missing), 404 Not Found, 429 Too Many Requests (rate limit), 500 Server Error
Content-Type	application/json — most common for IoT REST APIs. application/x-www-form-urlencoded (ThingSpeak). text/plain for simple values.
Authorization	API Key in header (X-API-Key: abc123), Bearer Token (Authorization: Bearer <JWT>), or Basic Auth.

3.3.2 HTTP IoT Example — Sending Data to ThingSpeak

```
// ESP32 sending sensor data to ThingSpeak via HTTP GET
#include <WiFi.h>
#include <HTTPClient.h>

const String apiKey    = "YOUR_THINGSPEAK_WRITE_API_KEY";
const String serverURL = "http://api.thingspeak.com/update";

void sendToThingSpeak(float temperature, float humidity) {
  if (WiFi.status() != WL_CONNECTED) return;
  HTTPClient http;
  String url = serverURL + "?api_key=" + apiKey
    + "&field1=" + String(temperature)
    + "&field2=" + String(humidity);
  http.begin(url);                                // Start connection
```

```

int httpCode = http.GET(); // Send GET request
if (httpCode == 200) {
    Serial.println("Data sent! Response: " + http.getString());
} else {
    Serial.println("HTTP Error: " + String(httpCode));
}
http.end(); // Free resources
}

```

3.3.3 Comparison: HTTP vs MQTT vs CoAP — For IoT

Property	HTTP/HTTPS	MQTT	CoAP
Protocol Type	Request-Response	Publish-Subscribe	Request-Response (REST)
Transport	TCP	TCP	UDP
Header Size	Large (200–2000 bytes)	Very small (2 bytes min)	Tiny (4 bytes min)
Broker/Middleware	Not required (REST API)	Required (broker)	Not required
Real-time Push	No (polling needed)	Yes (native pub/sub)	Via Observe extension
Security	TLS/SSL (HTTPS)	TLS/SSL (port 8883)	DTLS
Power Impact	High (full TCP + HTTP)	Low	Very Low
Firewall/NAT	Works well (standard port 80/443)	Usually works (port 1883)	May have NAT issues (UDP)
IoT Device Fit	Capable devices (ESP32+, RPi)	ALL IoT devices	Constrained MCUs
Use Case	Cloud REST APIs, dashboards	Telemetry, real-time monitoring	Device-to-device LAN

4. Network and Application Layer Protocols in IoT

4.1 IoT Protocol Stack — Mapping to OSI Model

Understanding where IoT protocols fit in the OSI (Open Systems Interconnection) model and TCP/IP stack is essential for a complete understanding of IoT networking.

OSI Layer	TCP/IP Layer	IoT Protocols & Technologies	Function
7 — Application	Application	MQTT, CoAP, HTTP/HTTPS, AMQP, WebSocket, XMPP, LWM2M	End-user data exchange, application logic
6 — Presentation	Application	JSON, XML, CBOR, TLS/SSL encryption	Data formatting, encryption, compression
5 — Session	Application	TLS/DTLS session management, WebSocket sessions	Session establishment and management
4 — Transport	Transport	TCP (MQTT, HTTP, WebSocket), UDP (CoAP, DNS)	End-to-end data delivery, port addressing
3 — Network	Internet	IPv4, IPv6, 6LoWPAN, RPL routing protocol	Logical addressing (IP), routing between networks
2 — Data Link	Network Access	IEEE 802.11 (Wi-Fi), IEEE 802.15.4 (Zigbee/6LoWPAN), Ethernet, LoRa MAC, BLE Link Layer	Node-to-node data transfer, MAC addressing
1 — Physical	Network Access	Wi-Fi radio, Zigbee radio, LoRa radio, Ethernet cable, GPIO, UART, SPI, I2C	Bits on the physical medium

4.2 IPv6 and 6LoWPAN in IoT

4.2.1 Why IPv6 is Critical for IoT

- **IPv4 Address Exhaustion:** IPv4 provides only 4.3 billion unique addresses — already exhausted. With tens of billions of IoT devices, IPv4 is completely insufficient.
- **IPv6 Address Space:** IPv6 provides 2^{128} addresses = 340 undecillion (3.4×10^{38}) — enough for every atom on Earth to have an IP address. Every IoT device can have a globally routable address.
- **IPv6 Features for IoT:** Auto-configuration (SLAAC — Stateless Address Autoconfiguration), multicast support, no need for NAT, built-in IPsec support, neighbor discovery.

4.2.2 6LoWPAN — IPv6 over Low-Power Wireless Personal Area Networks

6LoWPAN — Definition

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) is an adaptation layer (RFC 4944) that enables IPv6 packets to be transmitted over IEEE 802.15.4 wireless networks (like Zigbee). It compresses IPv6 headers (40 bytes → as few as 2 bytes) and fragments large IPv6 packets to fit within the 802.15.4 maximum frame size of 127 bytes.

- **Header Compression:** IPv6 header normally 40 bytes — 6LoWPAN compresses to 2–3 bytes for local addresses
- **Fragmentation:** IPv6 minimum MTU = 1280 bytes; 802.15.4 max frame = 127 bytes. 6LoWPAN fragments large packets into multiple small frames.
- **Mesh-Under vs Route-Over:** Mesh-Under: mesh routing at 6LoWPAN layer. Route-Over: routing at IP layer using RPL protocol.
- **Importance:** 6LoWPAN allows full IP connectivity to tiny battery-powered sensor nodes — bridging constrained sensor networks with the IP Internet directly.

4.3 DNS, DHCP, and NTP in IoT Context

Protocol	Role in IoT
DNS (Domain Name System)	Resolves human-readable names (broker.hivemq.com) to IP addresses. IoT devices use DNS to find cloud servers/brokers. Uses UDP port 53. mDNS (Multicast DNS) used for local IoT device discovery (service.local names).
DHCP (Dynamic Host Config)	Automatically assigns IP address, subnet mask, gateway, and DNS server to IoT devices when they connect to a network. IoT gateways often run DHCP servers for local sensor networks. Most IoT devices use DHCP by default.
NTP (Network Time Protocol)	Synchronizes device clocks to accurate internet time. Critical for IoT: timestamps on sensor data must be accurate, TLS certificates require correct time, log correlation across multiple devices requires synchronized clocks. IoT devices use pool.ntp.org servers.
mDNS / DNS-SD	Multicast DNS and DNS Service Discovery — allows IoT devices to advertise and discover services on local network without a central DNS server. Used by Apple HomeKit, Google Cast, Chromecast, Philips Hue bridge discovery.

5. Node-to-Gateway-to-Cloud Communication

The Node-to-Gateway-to-Cloud (N-G-C) communication model is the standard three-tier architecture for IoT deployments. It defines how data flows from physical sensor nodes, through local gateways, to cloud platforms for processing and storage.

5.1 The Three-Tier IoT Communication Architecture

Tier	Description, Protocols & Examples
 Tier 1 — NODES (Sensing/Actuating Layer)	Physical IoT devices — sensors, actuators, microcontrollers. • Examples: Arduino + DHT11, ESP32 + soil sensor, Raspberry Pi camera, Zigbee temperature sensor. • Protocols to Gateway: Zigbee, BLE, Z-Wave, Modbus, UART, SPI, I2C, LoRa, Wi-Fi. • Characteristics: Resource-constrained (limited CPU, RAM, battery), produce raw data, often cannot connect to Internet directly. • Data generated: Temperature readings, motion events, gas levels, images, GPS coordinates.
 Tier 2 — GATEWAY (Edge Layer)	Local hub/router that collects, aggregates, and forwards data from nodes to cloud. • Examples: Raspberry Pi, industrial IoT gateway (Sierra Wireless, Advantech), ESP32 (simple gateway), cloud-managed gateway (AWS Greengrass, Azure IoT Edge). • Functions: Protocol translation (Zigbee/BLE → MQTT/HTTP), data filtering and aggregation, local decision-making (edge computing), buffering data when cloud is unreachable. • Security: First line of defense — authentication of nodes, encrypting data before cloud transmission. • Protocols to Cloud: MQTT, HTTPS/REST, AMQP — all over standard IP networking (Wi-Fi/Ethernet/4G).
 Tier 3 — CLOUD (Processing & Application Layer)	Remote servers that receive, store, process, and visualize all IoT data at scale. • Examples: AWS IoT Core, Azure IoT Hub, Google Cloud IoT Core, ThingSpeak, IBM Watson IoT, Blynk. • Functions: Data ingestion at massive scale, time-series database storage, real-time analytics, ML model training, dashboard visualization, alerting/notifications. • User Interface: Web dashboards, mobile apps, REST APIs for third-party integration. • Protocols: Receives MQTT (most common), HTTPS REST, AMQP from gateways.

5.2 Data Flow — Complete End-to-End Example

Complete IoT Data Flow: Smart Greenhouse Example

STEP 1 — SENSING (Node): Soil moisture sensor reads 30% humidity. DHT22 reads 28°C, 65% RH. Arduino collects readings.

STEP 2 — NODE→GATEWAY: Arduino sends data over Zigbee to Raspberry Pi gateway. Protocol: Zigbee 802.15.4, 250 kbps.

STEP 3 — EDGE PROCESSING (Gateway): Raspberry Pi receives raw sensor values. Runs threshold check: if soil_moisture < 40%, set irrigation_flag = True. Formats data as JSON: {"temp":28, "humidity":65, "soil":30, "irrigate":true}

STEP 4 — GATEWAY→CLOUD: Raspberry Pi publishes JSON to MQTT broker (AWS IoT Core) on topic 'greenhouse/sensor01/data'. TLS encrypted on port 8883.

STEP 5 — CLOUD PROCESSING: AWS IoT Rule triggered. Data stored in DynamoDB time-series table. If irrigate=true: AWS Lambda triggers SNS notification to farmer's phone.

STEP 6 — CLOUD→GATEWAY→NODE (Control): Farmer approves irrigation via app. App publishes MQTT message 'greenhouse/actuator/valve ON'. Gateway receives, forwards to Arduino. Arduino activates solenoid valve relay.

STEP 7 — VISUALIZATION: Grafana dashboard shows temperature/humidity graphs for past 30 days. Alert sent if temperature > 35°C.

5.3 Edge Computing in IoT

Edge Computing — Definition

Edge computing moves data processing and computation CLOSER to where data is generated (at the edge of the network — on the gateway or even on the device itself), rather than sending all data to a distant cloud server. This reduces latency, bandwidth usage, and cloud processing costs.

Cloud Computing (Traditional)	Edge Computing
All data sent to cloud for processing	Data processed locally at gateway or device
High latency (round-trip to cloud)	Very low latency (local processing)
Requires constant internet connectivity	Works offline — stores data until reconnected

High bandwidth usage (all raw data sent)	Only processed results/summaries sent to cloud
Lower device cost (dumb nodes)	Smarter, more capable gateway required
Examples: Simple Arduino → HTTP POST to ThingSpeak	Examples: RPi running TF Lite model, AWS Greengrass on gateway

6. Quick Revision Summary & Important Exam Questions

6.1 Quick Revision Table — All Unit III Concepts

Topic	Key Points to Remember
UART	Asynchronous serial. 2 wires (TX+RX). Point-to-point. Baud rate must match. Frame = START+8 data bits+STOP. Common rates: 9600, 115200 bps.
SPI	Synchronous serial. 4 wires (MOSI, MISO, SCK, SS). Full-duplex. Fastest. No addressing — SS pin per device. Modes 0–3 (CPOL, CPHA).
I2C	Synchronous serial. 2 wires (SDA+SCL). 7-bit addressing (127 devices). Needs 4.7kΩ pull-ups. Wire.h library. Arduino: A4=SDA, A5=SCL.
Bluetooth Classic	BR/EDR, 2.4GHz, 1–3Mbps, ~100m, piconet (1 master + 7 slaves), FHSS (79 channels, 1600 hops/sec).
BLE	Bluetooth 4.0+. Ultra-low power (months on battery). GATT profiles. Central/Peripheral roles. BLE 5.0: up to 400m, 2Mbps.
Wi-Fi	IEEE 802.11. High bandwidth. 2.4GHz (longer range) and 5GHz (faster). STA/AP/dual mode on ESP32. WPA2/WPA3 security.
Zigbee	IEEE 802.15.4. 2.4GHz. 250kbps. Mesh network. 3 roles: Coordinator, Router, End Device. 65000 devices. Very low power. Smart home automation.
LoRa/LoRaWAN	CSS modulation. Sub-GHz bands. Range 10–15km rural. Ultra-low power (years on battery). Low data rate (up to 50kbps). Spreading Factor SF7–SF12.
LoRaWAN Classes	Class A: lowest power (2 RX after TX). Class B: scheduled RX. Class C: always listening (highest power).
MQTT	Pub/Sub, TCP, port 1883 (8883 TLS). Broker routes messages by topic. QoS 0 (fire & forget), 1 (at least once), 2 (exactly once).
MQTT Topics	Hierarchical string. '+' = single-level wildcard. '#' = multi-level wildcard (must be last character).
MQTT Features	Retained message = broker stores last message for new subscribers. LWT = broker publishes if client unexpectedly disconnects.
CoAP	RESTful, UDP, port 5683. 4-byte header. Methods: GET/POST/PUT/DELETE. CON/NON/ACK/RST message types. Observe for push. DTLS for security.
HTTP/HTTPS	Request-Response, TCP, port 80/443. Large headers. Standard for cloud REST APIs. Polling for IoT updates. Best for capable devices.
OSI Layers	App=MQTT/CoAP/HTTP, Transport=TCP/UDP, Network=IPv4/IPv6/6LoWPAN, Data Link=802.11/802.15.4, Physical=radio/cable
6LoWPAN	IPv6 over 802.15.4. Compresses 40-byte IPv6 header to 2 bytes. Fragments packets for 127-byte 802.15.4 frames. Enables IP connectivity for Zigbee networks.

Node-Gateway-Cloud	3 tiers: Nodes (sensing/actuating), Gateway (protocol translation, edge computing), Cloud (storage, analytics, visualization)
Edge Computing	Processing at gateway/node — reduces latency, bandwidth, cloud costs. Enables offline operation.
LPWAN Comparison	LoRaWAN (unlicensed, 15km, years battery) vs NB-IoT (licensed cellular) vs Sigfox (ultra-simple, 50km)

6.2 Important Exam Questions with Answer Outlines

2-Mark Questions

Q1. Differentiate between UART, SPI, and I2C protocols.

Answer: UART: 2 wires, async, point-to-point, baud rate matching. SPI: 4 wires, sync, full-duplex, fastest, SS per device. I2C: 2 wires, sync, 127 devices via 7-bit addressing, needs pull-up resistors.

Q2. What is the difference between Bluetooth Classic and BLE?

Answer: Classic: 1–3Mbps, high power, continuous streaming, piconet. BLE: very low power (months on battery), periodic small data, GATT profiles, IoT-optimised.

Q3. What are the three roles in a Zigbee network?

Answer: Coordinator (1 per network, manages network), Router (mains-powered, forwards messages, extends range), End Device (battery-powered, sleeps, leaf node only).

Q4. What are MQTT QoS levels? Explain each briefly.

Answer: QoS 0: At most once — no guarantee. QoS 1: At least once — guaranteed but may duplicate. QoS 2: Exactly once — guaranteed, no duplicates (4-way handshake).

Q5. What is CoAP? How does it differ from HTTP?

Answer: CoAP: RESTful protocol over UDP, 4-byte header, designed for constrained IoT devices. HTTP: over TCP, large headers (200+ bytes), designed for full-capability web clients.

Q6. Explain the Node-to-Gateway-to-Cloud architecture.

Answer: 3 tiers: Nodes (sensors/actuators, local protocols like Zigbee/BLE), Gateway (protocol translation, edge processing, forwards via MQTT/HTTPS), Cloud (storage, analytics, dashboards, user apps).

4-Mark / 5-Mark Questions — Answer Outlines

Q1. Explain MQTT protocol in detail — architecture, QoS levels, topics, retained messages, and LWT.

Answer outline — cover these points:

1. Architecture: Publisher → Broker → Subscriber. Broker routes by topic. TCP port 1883 (8883 TLS).
2. Topics: Hierarchical (home/bedroom/temp). Wildcards: + (single level), # (multi-level, last char).
3. QoS 0: fire & forget. QoS 1: at least once (PUBLISH+PUBACK). QoS 2: exactly once (4-step handshake).
4. Retained Message: broker stores last message per topic; new subscribers get it immediately on subscribe.
5. LWT (Last Will & Testament): broker publishes on behalf of client if it unexpectedly disconnects.
6. Implementations: Mosquitto (open-source broker), HiveMQ, AWS IoT Core, PubSubClient library (Arduino).

Q2. Compare MQTT, CoAP, and HTTP for IoT applications.

Answer outline — cover these points:

1. MQTT: Pub/Sub, TCP, broker required, QoS 0/1/2, tiny header, great for telemetry and real-time IoT.
2. CoAP: RESTful, UDP, no broker, 4-byte header, CON/NON messages, Observe extension, DTLS, best for LAN device-to-device.
3. HTTP: Request-Response, TCP, large headers, polling needed, best for capable devices and cloud REST APIs.
4. Comparison table: transport, broker needed, header size, real-time push, security, best use case.

Q3. Explain LoRa and LoRaWAN architecture, device classes, and compare with other LPWAN technologies.

Answer outline — cover these points:

1. LoRa: Physical layer — CSS modulation, SF7–SF12, sub-GHz bands, range 2–15km.
2. LoRaWAN: MAC protocol — End Node → Gateway → Network Server → Application Server.
3. Class A: lowest power, 2 RX windows after TX only. Class B: scheduled RX. Class C: always listening.
4. Security: AES-128 at both network (NwkSKey) and application (AppSKey) layers. OTAA vs ABP activation.
5. Compare: LoRaWAN (unlicensed, free, long range) vs NB-IoT (cellular, licensed) vs Sigfox (ultra-simple).

Q4. Explain Node-to-Gateway-to-Cloud architecture with a detailed real-world example.

Answer outline — cover these points:

1. Tier 1 Nodes: Sensor devices (Arduino, Zigbee sensors) — collect physical data.
2. Tier 2 Gateway: Raspberry Pi — protocol translation (Zigbee→MQTT), edge processing, buffering.
3. Tier 3 Cloud: AWS IoT Core — data ingestion, storage (DynamoDB), analytics, dashboards, notifications.
4. Real example: Smart greenhouse — soil sensor (Zigbee) → RPi gateway → AWS IoT (MQTT TLS) → Lambda → SNS alert → farmer's phone → farmer sends 'irrigate' command → gateway activates solenoid valve.
5. Edge computing benefit: RPi checks thresholds locally — only sends alerts to cloud (reduces bandwidth and latency).

Key Points to Remember

- UART = async, 2 wires, point-to-point | SPI = sync, 4 wires, fastest, SS per device | I2C = sync, 2 wires, 127 devices, needs pull-ups
- Bluetooth Classic = streaming audio/data; BLE = low power IoT sensors (months on battery)
- Zigbee: 3 roles = Coordinator (1) + Router (mains) + End Device (battery). Mesh. 65000 nodes.
- LoRaWAN: SF7 = fast short range; SF12 = slow very long range (15km). Class A = lowest power.
- MQTT QoS: 0=at most once (no ACK), 1=at least once (PUBACK), 2=exactly once (4-way)
- MQTT wildcards: + = one level, # = all remaining levels (must be last character)
- CoAP: UDP + 4-byte header + RESTful. HTTP: TCP + large headers. MQTT: TCP + pub/sub.
- N-G-C: Nodes (local protocols) → Gateway (protocol translation + edge computing) → Cloud (analytics + UI)